

Verified Tensor Operators for Safety-Critical ML: From Specification to Reference Implementation

João Machado[1,2], Ricardo Silva[1,2], Loïc Correnson[4], João Galego[2,5], Eric Jenn[3], Hugo Daniel Macedo[1], Jean Souyris[6], and Jorge Sousa Pinto[1(✉)]

[1] INESC TEC & Universidade do Minho, Braga, Portugal
jsp@di.uminho.pt
[2] Critical Software, Lisbon, Portugal
[3] IRT Saint-Exupéry, Toulouse, France
[4] CEA LIST, Saclay, France
[5] ISEG Executive Education, Lisbon, Portugal
[6] Airbus Commercial, Toulouse, France

Abstract. Safety-critical deployment of machine learning models requires unambiguous specification and verified implementation of tensor operators. This paper presents a formally verified workflow based on the Why3 deductive verification platform. The workflow takes each operator from an abstract WhyML specification over mathematical tensors to extracted C code, via a concrete implementation on flat arrays linked by a machine-checked refinement proof. A reusable library supports the development, providing abstract tensor types, a row-major layout with proven bijectivity, and a refinement mapping to a C-level representation. We illustrate the workflow on several ONNX operators, developed within the SONNX Working Group. Finally, we demonstrate how the abstract specifications compose to enable contract-based verification of functional properties of complete networks. The methodology is applicable beyond ONNX to any framework requiring verified tensor implementations.

1 Introduction

Machine learning (ML) models deployed in safety-critical systems must satisfy stringent requirements on traceability and correctness. Regardless of the framework used to train or exchange a model, be it ONNX [20], NNEF [23], TensorFlow [1], or a proprietary format, the semantics of its constituent tensor operators must be completely and unambiguously defined, and any implementation must be demonstrably faithful to that specification. Emerging standards in domains such as aeronautics [6,7,10] are making these expectations explicit, requiring applicants to capture a precise description of the trained model and to provide evidence that it is preserved through implementation.

In practice, however, the specifications provided by mainstream ML interchange formats fall short of this goal. Operator documentation is often minimal

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2027
G. Avni and C. Schilling (Eds.): SAIV 2026, LNCS 16831, pp. 36–58, 2027.
https://doi.org/10.1007/978-3-032-32357-6_2

or ambiguous, convenience features introduce implicit transformations, and certain behaviors are left to the runtime. Empirical studies confirm that these gaps lead to real divergences across implementations [12]. The problem is not specific to any single format: it reflects a fundamental mismatch between specifications and the level of rigor demanded by critical-systems engineering.

The Problem at a Glance. Figure 1 depicts the conventional ONNX development workflow. An ONNX model is a computational graph, whose operator semantics are defined only by informal descriptions and reference Python implementations. Relying on implicit conventions, production runtimes then re-implement each operator independently, and resort to testing to ensure consistency. This workflow provides no formal link between the specification and any concrete implementation, making it difficult to produce the certification evidence demanded by emerging standards.



Fig. 1. Conventional ONNX deployment workflow.

The SONNX Safe Workflow. Figure 2 shows the SONNX workflow, supported by a reusable WhyML library [9]. Each operator is first informally specified through natural language descriptions, mathematical formulas, and execution examples. From these, an abstract formal specification is derived as a WhyML ghost function over mathematical tensors, capturing the operator semantics independently of any implementation and ensuring traceability to the informal specification. Subsequently, a lower-level concrete formalization is introduced that operates on flat arrays in a C-compatible memory layout; a machine-checked refinement proof establishes that it faithfully implements the abstract specification (Specification phase). Finally, the verified code is automatically extracted into C via Why3's driver-based mechanism, producing a reference implementation/testbed that serves as a verified oracle for back-to back testing of production implementations (this work does not cover performance optimization), providing traceable evidence that the deployed code is consistent with the formal specification (Implementation phase). The activities represented by the dashed arrows in the figure are actively being developed, but fall outside the scope of this article.

We demonstrate the methodology on operators from the ONNX standard, taking advantage of the SONNX Working Group [13], an initiative developing a safety-related profile for critical systems (see Sect. 3). The choice of ONNX as the application domain provides a concrete and industrially relevant testbed, but the methodology and the workflow itself can be instantiated for any tensor-based computation framework whose operators need verified implementations.

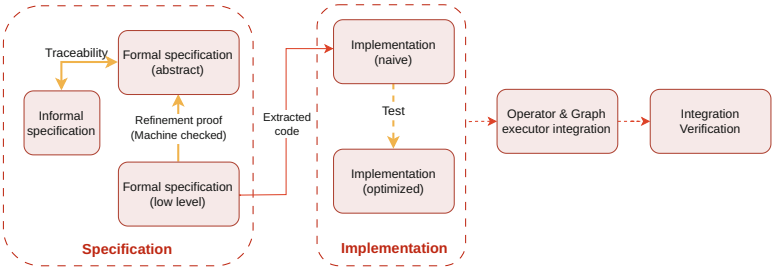


Fig. 2. The SONNX safe workflow. It replaces informal specifications with a two-stage formal development, producing a verified reference implementation that is linked to the abstract specification by a machine-checked refinement proof.

Beyond individual operators, we show that the abstract layer of the library also serves as a foundation for reasoning about complete models: by composing operator specifications, network-level properties can be stated as WhyML contracts and discharged by automated provers, without any involvement of the concrete layer or C code. Models can also be promptly implemented in C as *executable specifications*, using the reference implementations of its operators.

In summary, our contributions are:

1. A WhyML library for verified tensor operator development, providing abstract types, layout formalization, concrete representation, and refinement infrastructure to carry an operator from specification to extracted C code.
2. The complete development of an ONNX operator, as an illustrative example.
3. A report on the application of the workflow to further operators, with quantitative data on specification size, proof effort, and extraction results.
4. A demonstration of how the same abstract infrastructure supports model-level verification. This is illustrated by proving functional properties of a logistic regression network composed from verified operator specifications.

The paper is organized as follows. Section 2 discusses background and related work, and Sect. 3 summarizes the SONNX initiative, which provides the application context for our work. Section 4 describes the supporting WhyML library. Section 5 describes the ONNX *Flatten* operator, and Sects. 6 to 8 present the different stages of the formal development workflow: high-level specification, implementation/refinement, and extraction of reference C implementation. Section 9 reports on the full development of a set of operators, and the effort involved. Section 10 shows how the abstract operator specifications compose to enable verification of a complete model. Section 12 concludes the paper.

2 Background and Related Work

Why3 [9] is a platform for deductive program verification built around WhyML, a first-order language with polymorphic types, mutable variables, and ghost code.

Programs are annotated with specification elements such as preconditions, post-conditions, and invariants, from which Why3 generates verification conditions (VCs) discharged by automated provers (Alt-Ergo, Z3, CVC5) or interactive proof assistants (Coq/Rocq, Isabelle). Why3's extraction mechanism translates verified WhyML code into OCaml or C, erasing ghost code and mapping types and operations via user-defined drivers. This approach has been used to produce verified C libraries with performance competitive with hand-written code [18]. Our work applies a similar pattern systematically to tensor operators, exploiting Why3's built-in memory model for reasoning about pointers and flat arrays.

ONNX [20], initiated by Microsoft and Facebook, is the most widely adopted interchange format for ML models, supported by frameworks such as PyTorch and TensorFlow and by runtimes including ONNX Runtime. The NNEF format [23], developed by the Khronos Group, provides a well-defined and human-readable serialization with good documentation, but never achieved comparable adoption. Both formats share the limitation that their operator semantics are ultimately defined by their implementations rather than by a rigorous, independent specification. Jajal et al. [12] conducted a large-scale study of failures in ONNX model converters, finding that a significant proportion of defects are attributable to specification ambiguity. These findings motivate the need for formal, implementation-independent operator specifications.

The integration of ML into safety-critical products is the subject of active standardization. In aeronautics, AS6983/ED-324 [10] defines a process in which the ML Model Description serves as the pivot artifact between training and implementation. EASA's Concept Paper [6] and its proposed detailed specifications [7] formulate objectives requiring applicants to capture and validate the model description. These standards create a regulatory incentive for formal artifacts of the kind produced by our workflow, regardless of which interchange format is used to describe the model. In the next section we describe how The SONNX Working Group [13] is developing a safety-related profile for the ONNX standard, targeting the needs of critical systems.

Finally, the verification of neural networks has been an active research area. Liu et al. [17] provide a comprehensive survey of verification algorithms for deep neural networks, including SMT- and LP-based approaches such as Reluplex [14] and Planet [8], which became foundational methods for the verification of ReLU neural networks. Huang and colleagues [11] survey safety and trustworthiness aspects including verification, testing, and adversarial robustness. Zhang et al. [26] survey robustness assessment techniques for image recognition networks. Other advances in neural network verification have introduced scalable bound-propagation and SMT-based tools such as CROWN [25], which improves robustness certification via efficient convex relaxations, and Marabou [24], which extends SMT-style reasoning for piecewise-linear networks through a highly optimized constraint-solving framework. These approaches, which rely primarily on constraint solving, abstract interpretation, and bound propagation, verify properties of trained networks (such as robustness to adversarial perturbations) but do not employ deductive verification. Our work is complementary

to network-level verification, which assumes correct operator implementations – our methodology provides them using deductive verification techniques. We also demonstrate (Sect. 10) how the same framework may support reasoning about network-level functional properties.

3 Application Context: The SONNX Profile

We demonstrate our methodology within the SONNX (Safety-related ONNX) Working Group, which provides both the operator specifications we formalize and the industrial validation context. This section summarizes the relevant aspects of the initiative; a full account is given by Jenn et al. [13]. The SONNX Working Group was established within the ONNX community in September 2024. It brings together participants from industry (Airbus, Thales, Collins, Bosch, Critical Software) and research institutions (CEA, ONERA, INRIA, IRT Saint-Exupéry), and meets biweekly. All work products are publicly available on the ONNX repository[1]. The SONNX profile is governed by several design principles: full compatibility with ONNX (restrictions on operator usage are allowed, semantic changes are not); mandatory explicit specification of all attributes, eliminating defaults; prohibition of implicit computations such as shape inference; and deterministic behavior. It currently targets a subset of operators selected to support industrial use cases including YOLOv8[2] and MobileNetSSDv2 [3], with a first complete version expected by the end of 2026.

The goal of the profile is to provide, for each operator, an informal specification, a formal specification in WhyML, and a reference C implementation. Jenn et al. outlined four stages for the specification and implementation phases of the SONNX workflow, that stand at the core of the present paper:

1. *Abstract formal specification*: a function over mathematical tensors, expressing the operator's intended semantics independently of implementation.
2. *Concrete implementation*: a WhyML function operating on a C-compatible tensor representation (rank, dimension array, flat data pointer).
3. *Refinement proof*: a machine-checked proof that the concrete implementation is correct with respect to the abstract specification.
4. *C extraction*: automatic translation of the verified code into C via Why3's driver-based extraction mechanism, producing the reference implementation.

The latter implementation plays a critical role as a verified oracle for back-to-back testing of production implementations, enabling developers to validate that performance-oriented code (which may employ target-specific optimizations) produces results consistent with the formal specification. In certification contexts, this testing approach provides evidence that the deployed implementation faithfully realizes the specified behavior, addressing a key requirement of

[1] https://github.com/ericjenn/working-groups/tree/ericjenn-srpwg-wg1/safety-related-profile/sonnx.

[2] https://github.com/ultralytics/ultralytics.

standards. The reference implementation is deliberately kept simple and traceable to the specification, precisely to serve this validation role.

The remainder of this paper describes how this methodology is realized in practice: we present the library infrastructure that supports the workflow, describe the complete formal development methodology (including refinement proofs and extraction) using the *Flatten* operator as worked example, and report on its successful application to a broader set of ONNX operators. We additionally demonstrate how the abstract layer of the development serves as a foundation for the deductive verification of functional properties of complete networks.

4 The WhyML Tensor Library

The formal development is supported by a set of WhyML modules organized in layers: abstract specifications of tensors and their properties, a row-major memory layout with proven bijectivity, a concrete C-level representation with a refinement mapping to the abstract level, and extraction drivers that translate the verified code into C. We describe each layer in turn.

Abstract Tensors. The module `tensor.Range` formalizes multi-dimensional index ranges. A shape is represented as a list of integers; the total number of elements is given by the product of dimensions. The predicate `valid ks ds` holds when `ks` is a multi-dimensional index within the bounds prescribed by shape `ds`; `positive ds` asserts that all dimensions in a shape are strictly positive; and the function `size ds` computes the product of dimensions, with the convention that a zero-dimensional shape corresponds to a scalar. The module `tensor.Tensor` defines the abstract tensor type (Listing 4.1). A tensor is a record comprising a shape, a data function mapping multi-dimensional indices to values, and a background value. The first invariant ensures the shape has strictly positive dimensions; the second requires that the data function returns the background value on any index that is not valid, which enables a key extensionality result:

```
type data 'a = map (list int) 'a
type tensor 'a = {
  dims : list int ;
  data : data 'a ;
  background : 'a ;
}
invariant { positive dims }
invariant { forall k. valid k dims \/ data k = background }
```

Listing 4.1. Abstract tensor type (`tensor.Tensor`)

```
predicate (==) (a b : tensor 'a) = a ~= b /\ forall k. valid k a.dims -> a.data k = b.data k
lemma exteq: forall a b : tensor 'a. a == b <-> a = b
```

The predicate `==` asserts that two tensors have the same dimensions and the same background using the predicate denoted by `~=` . Additionally, it checks that they agree on all *valid* indices. The lemma `exteq` states that this is equivalent to primitive equality (built-in `=`). The type invariant is what makes this

work: since both tensors must return their (common) background value on every invalid index, agreement on valid indices suffices to establish that the data functions are equal everywhere. This lemma plays a central role in refinement proofs (Sect. 7), where pointwise equality on valid indices (typically established by a loop invariant) needs to be lifted to full tensor equality to discharge the post-condition. It is equally important for reasoning compositionally about complete networks (Sect. 10).

Row-Major Layout. The module `layout.CFlat` formalizes the row-major layout as a pair of pure functions on lists of integers: `offset`, which maps a multi-dimensional index and a shape to a flat integer position, and `index`, which performs the inverse operation. These two functions form a bijection between valid multi-dimensional indices and flat positions in $[0, \texttt{size}\ ds[$. The module also provides several key lemmas. Although `offset` and `index` operate on abstract lists with no dependence on pointers or memory, they play a dual role in the development. At the abstract specification level, they allow for index reorganization: the specification of `Flatten`, for instance, defines its data mapping as a composition of `offset` and `index` across different shapes, relating output and input indices through the flat positions (see Sects. 5 and 6). At the concrete level, the same functions describe how multi-dimensional tensors are physically laid out in flat C arrays, and the bijection lemmas are used in refinement proofs to connect the abstract data function to the contents of the underlying memory.

C Memory Model and Machine Types. The concrete layer of the development rests on the modules `std.Clib` and `std.Cfloat`. `Clib` is the foundation of all C-level reasoning: it wraps Why3's lightweight C memory model (`mach.c.C`), which provides a `ptr 'a` type representing C pointers equipped with zone, offset, and bounds information, and builds on it with additional predicates and operations for working with arrays in memory (Listing 4.2).

```
predicate valid_range (vp : ptr 'a) (p q : int) =
  q <= p \/ ( 0 <= vp.min <= vp.offset /\ 0 <= p <= q <= max_int32 /\
              0 <= vp.min <= vp.offset + p /\ vp.offset + q <= vp.max <= vp.plength )

let ghost function value_at (vp : ptr 'a) : map int 'a =
  pure { fun k -> vp.data.Array.elts (vp.offset + k) }
```

Listing 4.2. C pointer abstraction (`std.Clib`)

The predicate `valid_range` asserts that a contiguous range of positions within a pointer's backing array is accessible. The ghost function `value_at` extracts the logical content of a pointer as a map from integer offsets to values, bridging the C memory model and the pure logical world. The module also provides functions that convert contiguous segments of a C array into logical lists, enabling statements such as "the first n elements of array `u` form the list `ds`". These are central to `libvector.CIndex` (see below), where C `int32` arrays are converted into lists of integers.

The module `std.Cfloat` defines an abstract `float` type with arithmetic operations whose postconditions are expressed over $\mathbb{R}$, equating the result of each floating-point operation with the corresponding exact real arithmetic operation. This means that all specifications reason over $\mathbb{R}$, abstracting away rounding errors. At extraction time, the driver maps `float` to C `double` and each operation to its C counterpart. This design cleanly separates the question of numerical accuracy (which the SONNX profile addresses through separate error bound analyses) from the question of functional correctness of the operator implementations, which is what the present formal development targets. Finally, auxiliary extensions to the Why3 standard library are provided in the modules `std.Int` and `std.Lists`, which includes the `get_dim` accessor for dimension lists.

Concrete Tensors and the Refinement Mapping. The next two modules are meant to be extracted to C, and compiled together with the different operator functions. `libvector.CIndex` provides C-level operations on arrays of `int32` values, used to represent dimension and coordinate arrays. Its key function `coffset` computes the flat offset from coordinate and dimension arrays, with a loop whose invariant links the result back to the abstract `offset` function through a coercion `ivector` that converts a pointer-and-length pair to a list of integers. This module also provides `cdim_size`, which computes the total number of elements from a dimension array. The module `libtensor.CTensor` (Listing 4.3) defines the concrete tensor type (a C-level structure) and the `tensor` function, the *refinement mapping* that constructs an abstract `tensor` from a concrete `ctensor`. The abstract data is read from the flat data array via `tensor_value_at`, which delegates to `value_at` and `offset`, and returns `0.0` for invalid indices, matching the background value. Since `tensor` is declared `ghost`, it exists only for specification and proof purposes, and is erased during extraction. The `valid_tensor` predicate collects the well-formedness conditions for a concrete tensor: the dimension array must have positive entries with a total size within `int32` bounds, and the data array must be a valid and writable memory region of sufficient size.

The refinement pattern for each operator follows a uniform scheme using the refinement mapping. A low-level function takes concrete tensors as arguments and includes a postcondition of the form
`ensures { tensor output = op_abstract (tensor x) (tensor w) ... }`.
The implementation operates on the flat data arrays via index arithmetic, with a loop invariant asserting pointwise equality on entries processed so far. After the loop, an assertion using extensional equality is established from the loop invariant, and lifted to the logical equality required by the postcondition.

Extraction to C. The final stage translates the verified WhyML code into C using Why3's extraction mechanism, guided by driver files. The driver `cdriver.drv` (Listing 4.4) maps WhyML types and operations to their C equivalents, arithmetic operations to their C syntax with appropriate precedences, array access to bracket notation, and the `ctensor` record to a C `struct`. Ghost functions, proof assertions, and invariants are erased, leaving only executable code. Extraction is performed modularly: Why3 is invoked to extract the `libvector` and `libtensor`

```
type farray = ptr float
type ctensor = {
  t_rank : int32 ;
  t_dims : iarray ;
  t_data : farray ;
}

function tensor_value_at (k : list int) (t : ctensor) : real =
  if valid_index k t then value_at t.t_data (tensor_offset k t)
  else 0.0
function tensor_value (t : ctensor) : list int -> real =
  fun k -> tensor_value_at k t

let ghost function tensor (t : ctensor) : tensor real
  requires { valid_tensor t }
  ensures  { result.dims = tensor_dim t }
  ensures  { result.data = tensor_value t }
  ensures  { result.background = 0.0 }
=
  { dims = tensor_dim t ;
    data = tensor_value t ;
    background = 0.0 }
```

Listing 4.3. Concrete tensor type and refinement mapping (`libtensor.CTensor`)

modules as separate C compilation units. Operator modules are extracted in the same way, producing C functions that operate on `struct ctensor` values.

```
module std.Cfloat
  syntax type float    "double"
  syntax val ( .+ )    "%1 + %2"   prec 8 8 7
  syntax val ( .* )    "%1 * %2"   prec 7 7 6
  syntax val ( ./ )    "%1 / %2"   prec 7 7 6
  ...
end

module std.Clib
  syntax val ([])      "%1[%2]"    prec 1 1 15
  syntax val ([]<-)    "%1[%2] = %3"  prec 14 14 15 14
  ...
end
```

Listing 4.4. Extraction driver excerpt (`cdriver.drv`)

5 Example: Flatten Operator

Listing 5.1 contains the formal specification of the ONNX Flatten operator, which reshapes an input tensor of arbitrary rank into a two-dimensional tensor by collapsing dimensions around a given axis. Given a tensor X of shape $(d_0, \ldots, d_{r-1})$ and an axis parameter a with $0 \leq a < r$, the output is a tensor Y of shape (d_{Y_0}, d_{Y_1}) where $d_{Y_0} = \prod_{i=0}^{a-1} d_i$ and $d_{Y_1} = \prod_{i=a}^{r-1} d_i$, with the convention that an empty product equals 1. This preserves the total number of elements ($d_{Y_0} \cdot d_{Y_1} = \prod_i d_i$) and the row-major ordering of the data. Following ONNX conventions, a negative axis a is interpreted relative to the rank, normalized in the specification by the function $\texttt{normalize_axis}(a, r) =$ if $a < 0$ then $a + r$ else a.

```
module OPFlatten
  use tensor.Tensor, tensor.Range, layout.CFlat, (...)

  let rec ghost function calculate_dY0 (x : tensor 'a) (axis : int) (i : int) : int
    requires { 0 <= axis <= length x.dims }
    requires { 0 <= i <= axis }
    variant { axis - i }
  =
    if Int.(i >= axis) then 1
    else get_dim x.dims i * calculate_dY0 x axis Int.(i + 1)

  let rec ghost function calculate_dY1 (x : tensor 'a) (axis : int) (i : int) : int
    requires { 0 <= axis <= length x.dims }
    requires { axis <= i <= length x.dims }
    variant { length x.dims - i }
  =
    if Int.(i >= length x.dims) then 1
    else get_dim x.dims i * calculate_dY1 x axis Int.(i + 1)

  function flat_dims (x : tensor 'a) (axis : int) : list int
  = let dY0 = calculate_dY0 x axis 0 in
    let dY1 = calculate_dY1 x axis axis in
    Cons dY0 (Cons dY1 Nil)

  function normalize_axis (axis : int) (x_rank : int) : int
  = if axis < 0 then axis + x_rank else axis

  function dflatten (x : tensor 'a) (y_shape: list int) : data 'a
  = fun ks -> if valid ks y_shape then
                let y_coords_flat = offset ks y_shape in
                let x_coords = index y_coords_flat x.dims in
                x.data x_coords
              else x.background

  let ghost function flatten (x : tensor 'a) (axis : int) : tensor 'a
    requires { -length x.dims <= axis <= length x.dims }
  =
    let axis_normalized = normalize_axis axis (length x.dims) in
    let y_shape = flat_dims x axis_normalized in
    { dims = y_shape ; data = dflatten x y_shape ; background = x.background }
end
```

Listing 5.1. Flatten Operator: formal specification (abstract definition)

Output Shape Computation. The output dimensions are computed by functions `calculate_dY0` and `calculate_dY1`, which respectively multiply the input dimensions before and from the (normalized) axis onward. Both functions recursively traverse the dimension list of the input tensor using the `get_dim` accessor:

$$\texttt{calculate_dY0}(X, a, i) = \begin{cases} 1 & \text{if } i \geq a \\ d_i \cdot \texttt{calculate_dY0}(X, a, i{+}1) & \text{otherwise} \end{cases}$$

$$\texttt{calculate_dY1}(X, a, i) = \begin{cases} 1 & \text{if } i \geq r \\ d_i \cdot \texttt{calculate_dY1}(X, a, i{+}1) & \text{otherwise} \end{cases}$$

where r is the rank of X and d_i its i-th dimension. The output shape is then (d_{Y_0}, d_{Y_1}) with $d_{Y_0} = \texttt{calculate_dY0}(X, a, 0)$ and $d_{Y_1} = \texttt{calculate_dY1}(X, a, a)$.

Data Mapping. The core of the specification is the data function `dflatten`, which defines the value of the output tensor at each valid two-dimensional index. The mapping exploits the `offset` and `index` functions from the `CFlat` layout module (Sect. 4), which respectively convert a multi-dimensional index to a flat position in row-major order and vice versa. For a valid index ks in the output shape (d_{Y_0}, d_{Y_1}), the output value is determined as follows:

1. Compute the flat position $p = \texttt{offset}(ks,\ (d_{Y_0}, d_{Y_1}))$.
2. Recover the multi-dimensional input index $xs = \texttt{index}(p,\ (d_0, \ldots, d_{r-1}))$.
3. Return the input value $X[xs]$.

In WhyML, this is expressed as follows for valid ks: $\texttt{dflatten}(X, y_shape)(ks) = X.data\big(\texttt{index}(\texttt{offset}(ks, y_shape),\ X.dims)\big)$. The background value of X is returned otherwise. The correctness of this mapping relies on the fact that both the input and output shapes describe the same total number of elements, so the flat positions in $[0, d_{Y_0} \cdot d_{Y_1}[$ are in bijection with both the set of valid output indices and of valid input indices (the `CFlat` module provides lemmas establishing that `offset` and `index` are mutually inverse).

Top-Level Specification. The complete specification assembles these components into the function `flatten`, which takes a tensor X and an axis a, normalizes the axis, computes the output shape, and constructs the result tensor with the data function defined above and the same background value as the input:

$$\texttt{flatten}(X, a) = \big(\,(d_{Y_0},\ d_{Y_1}),\ \texttt{dflatten}(X, (d_{Y_0}, d_{Y_1})),\ X.background\,\big)$$

This specification is purely declarative: it defines what the output must be, without prescribing how to compute it. The output data function is defined compositionally in terms of the input data function and the layout bijection.

6 High-Level Specification in Why3 (Stage 1)

Starting from the informal specification, which describes the operator's behavior in natural language, mathematical formulae, and examples, an abstract formal specification is written in WhyML, the specification and programming language of the Why3 deductive verification platform [9]. At this level, tensors are modelled as purely mathematical objects, tuples consisting of a shape (a list of dimension sizes), a total function from indices to values, and a background value ensuring the function is defined everywhere on its domain. Operators are specified as functions over these abstract tensors, expressing the intended mathematical semantics directly. The abstract specification serves as the authoritative reference for what the operator computes, independent of implementation concerns.

Ghost Functions. Recall that in our methodology we specify operators formally by means of mathematical functions. In the `OPFlatten` module, `flat_dims`, `normalize_axis`, and `dflatten` are all examples of mathematical functions

defined in the logic language of Why3. The platform offers an alternative way to define such functions, in the form of *ghost functions*. A *let function* is a WhyML program function that is free of side effects (the *function* keyword is interpreted as *pure* function – *let* definitions missing that keyword may have side effects). Because they are pure, let functions can be used in both programs and logic, but if they are additionally marked as *ghost*, they become exclusively logic functions that cannot be used in programs. Ghost code is a very useful feature, allowing for specification elements to be manipulated within programs without interfering with their execution (which means that erasing these elements will not cause problems later, when programs are extracted to an external language).

For our purposes in this paper, ghost functions provide an alternative way to define mathematical functions. They can be equipped with contracts, which means that they can be made *partial* (by including a precondition), and also that their termination can be established using a *variant*, which is useful when termination is not proved automatically. In the `OPFlatten` module, the functions `calculate_dY0` and `calculate_dY1` must be written as ghost functions, since they require variants for termination. Their definitions make use of the `get_dim` let function, which includes preconditions that will trigger the generation of verification conditions (and thus the need for proofs) already at the level of the abstract specification. In order for these VCs to be discharged, the preconditions must be carried over to `calculate_dY0/dY1`, making these functions partial.

Another situation in which ghost definitions must be used is when defining mathematical functions that manipulate expressions of record types equipped with type invariants, as is the case of the tensor type. The function `flatten` in the `OPFlatten` module is the formal definition of the ONNX Flatten operator; since it returns a tensor, it must be written as a ghost function. Since this function is defined using `dflatten`, which in turn uses `calculate_dY0` and `calculate_dY1`, it must be equipped with a precondition framing the value of the `axis` parameter, to respect the preconditions of the latter functions.

Lemma Functions. Why3 generates verification conditions triggered by the type invariant of type `tensor`, including one requiring that the list of dimensions contains only positive elements. In order for this VC to be successfully proved, we must ensure that `flat_dims` returns a list of positive dimensions, which requires proving that the functions `calculate_dY0` and `calculate_dY1` have positive results. This is fairly obvious, but since the results are calculated recursively, an inductive argument is required. In Why3 (and similar auto-active tools) it would suffice to include the desired property in the function's contract as a *postcondition* in these functions. In our example, including the clause `ensures { 0 < result }` would cause Why3 to generate verification conditions appropriate to express the inductive argument. In the context of the methodology used in this paper, however, we will prefer to keep high-level specifications free from postconditions, since these are not required as specification elements, and including postconditions in certain cases causes Why3 to no longer encode the function definition in logic, keeping it hidden behind its contract (seen as an abstraction barrier, as is typical when reasoning about imperative programs).

This would imply that contracts have to describe the function in a *complete* way, which is not what we seek – our goal is to use mathematical functions, not contracts, as formal specifications for ONNX operators. The above lemma and its inductive proof can instead be given using a *lemma function*. For instance:

```
let rec lemma calculate_dY0_positive (x: tensor 'a) (axis: int) (i: int)
  requires { positive x.dims }
  requires { 0 <= axis <= length x.dims }
  requires { 0 <= i <= axis }
  ensures  { 0 < calculate_dY0 x axis i }
  variant  { axis - i }
=
  if i >= axis then ()  else calculate_dY0_positive x axis (i + 1)
```

This definition (i) introduces a lemma stating that the conjunction of the preconditions implies the postcondition, and (ii) generates proof obligations for this statement, using the definition as a script: if $i \geq axis$ then we are done, otherwise we use the inductive hypothesis that results from instantiating the lemma's parameters with x, $axis$, and $i + 1$. The exhaustive set of lemmas included in each module can be found in the accompanying repository.

7 Implementation and Refinement (Stages 2 and 3)

A second, more concrete representation of tensors is introduced in WhyML, closer to what a C implementation would manipulate. In this representation, a tensor is described by its rank, an array of dimension sizes, and a pointer to a flat one-dimensional data array storing the tensor's elements in row-major order. Each operator is then re-implemented as a WhyML function operating on these concrete tensors, typically using imperative constructs such as nested loops and mutable variables rather than the recursive definitions of the abstract level.

The connection between the two levels is established through a *refinement mapping* – a logical function that converts a concrete tensor into its abstract counterpart by reconstructing the shape and the index-to-value function from the flat array representation. The low-level WhyML function for each operator is equipped with a contract whose postcondition asserts that applying this refinement mapping to the concrete output yields exactly the result prescribed by the abstract specification. Why3's verification condition generator produces proof obligations from this contract, which are then discharged using automated provers (Alt-Ergo, Z3, CVC5) or, when necessary, interactive proof in Coq/Rocq or Isabelle. A successful proof guarantees that the low-level implementation is a correct refinement of the abstract specification.

A concrete implementation of Flatten is shown in Listing 7.1. It takes an input tensor `x`, a pre-allocated output tensor `r` of rank 2, and an axis parameter.

A Straightforward Implementation. The concrete implementation of the Flatten operator reduces to a flat copy of the data array. Because both the input and output tensors store their elements in row-major order, and flattening only reinterprets the shape without altering the ordering of elements, the i-th element of the input's data array is exactly the i-th element of the output's data array.

```
module COPFlatten
  use OPFlatten, tensor.Tensor, libtensor.CTensor, libvector.CIndex, std.Clib, mach.int.Int32

  let cflatten (x r : ctensor) (axis: int32)
    requires { valid_tensor x }
    requires { valid_tensor r }
    requires { r.t_rank = 2 }
    requires { let axis_normalized = normalize_axis (to_int axis) (length (tensor x).dims)
               in (ivector r.t_dims r.t_rank) = flat_dims (tensor x) axis_normalized }
    requires { vdim x.t_dims x.t_rank = vdim r.t_dims r.t_rank }
    requires { -length (tensor x).dims <= (to_int axis) <= length (tensor x).dims }
    ensures  { tensor r = flatten (tensor x) (to_int axis) }
  =
    let m = cdim_size r.t_dims r.t_rank in
    for i = 0 to m - 1 do
      invariant { forall k. 0 <= k < i -> value_at r.t_data k = value_at x.t_data k }
      r.t_data[i] <- x.t_data[i]
    done;
    assert { tensor r == flatten (tensor x) (to_int axis) }
end
```

Listing 7.1. Flatten Operator: concrete definition

The implementation is therefore a single loop:
`for i = 0 to m - 1 do r.t_data[i] <- x.t_data[i]`
where m is the total number of elements, computed from the output dimension array. No index arithmetic, axis manipulation, or layout transposition is needed at runtime; the entire complexity of the operator lies in ensuring that the output tensor has been set up with the correct shape before the copy takes place.

Contract and Refinement. The correctness of this implementation is expressed by the following refinement postcondition that makes use of the refinement mapping `tensor` and the abstract ghost function `flatten`:
`ensures { tensor r = flatten (tensor x) (to_int axis) }`.
The preconditions establish that the copy is semantically correct if:

- Both tensors are valid (well-formed dimension arrays and writable data regions).
- The output tensor has rank 2.
- The output dimensions match those computed by `flat_dims` on the input tensor and the normalized axis (the caller has already set up the shape (d_{Y_0}, d_{Y_1})).
- The total number of elements is the same in both tensors.

The loop invariant asserts that all entries copied so far coincide, so after the loop completes the data arrays are pointwise equal. A final assertion establishes extensional equality (`==`) between the concrete output, viewed through the refinement mapping, and the abstract specification, by connecting three facts: (i) the output dimensions are those prescribed by `flat_dims`, (ii) the flat data arrays are identical, and (iii) the layout functions `offset` and `index` (used in the abstract `dflatten`) compose to the identity on valid flat positions, so the abstract data mapping yields the same value as reading directly from the flat array.

The simplicity of this implementation illustrates a general point: a complex operator at the specification level, where the data mapping is defined through a composition of `offset` and `index` across different shapes, can have a trivial implementation when the storage layout is taken into account. The role of the

formal development is precisely to bridge this gap, providing a machine-checked proof that the naïve flat copy does implement the abstract reshape.

It is worth noting that the preconditions, which are not present at the abstract level, place the burden of computing the output shape on the caller. In a complete graph execution engine, the shape would be determined during a prior allocation phase. The operator implementation itself is then pure data movement, with all semantic content captured in the contract.

8 Extraction of Reference Implementation (Stage 4)

The final stage of the workflow extracts the verified low-level WhyML code into C, producing the SONNX reference implementation of the operator. Why3's extraction mechanism erases all specifications, invariants, assertions, and ghost elements, and maps WhyML types and operations to their C equivalents according to a set of *driver* files (for example, mapping the abstract tensor type to a C `struct`). The extraction is performed modularly, with each WhyML module producing a C file and a corresponding header file. The generated code is deliberately naïve, for instance, using straightforward nested loops for convolution rather than optimized algorithms such as im2col or Winograd, so it remains closely traceable to the formal specification. This reference implementation is not intended for deployment but serves as a verified oracle against which optimized, target-specific implementations can be tested in a back-to-back configuration.

The extracted C code for `Flatten` (Listing 8.1) is very concise. The generated function takes the input and output tensors as `struct ctensor` values and an `int32_t` axis parameter. It computes the total number of elements via `cdim_size` (extracted from `libvector.CIndex`) and copies the flat data array element by element. The `axis` parameter, which governs the output shape, is actually not used: the output tensor is assumed to have been allocated with the correct shape by the caller, the copy is valid because both tensors share the same row-major element ordering (see Sect. 7). The loop structure, an infinite `for` loop with a trailing `break` condition, is extracted from the WhyML `for` loop. The generated code links against the extracted library object files `cindex.o` and `ctensor.o`. It is worth noting that the generated header file `copflatten.h` declares a clean C interface depending only on `cindex.h` and `ctensor.h`, making the extracted operator straightforward to integrate into a larger runtime (or test framework).

```
void cflatten(struct ctensor x, struct ctensor r, int32_t axis) {
  int32_t m, i, o;
  m = cdim_size(r.t_dims, r.t_rank);
  o = m - 1;
  if (0 <= o) {
    for (i = 0; ; ++i) {
      r.t_data[i] = x.t_data[i];
      if (i == o) { break; }
    }}}
```

Listing 8.1. Extracted C code for the *Flatten* operator

9 A Gallery of Verified Tensor Operators

The accompanying repository[3] contains the developments corresponding to the application of the complete four-stage workflow to seven ONNX operators, spanning different categories of operations: element-wise arithmetic (`Add`, `Sigmoid`), shape manipulation (`Flatten`, `Reshape`), linear algebra (`MatMul`), and convolutional operations (`Conv2d`, `MaxPool2d`). Table 1 summarizes the development effort for each operator, reporting the number of specification elements (functions, contracts, assertions, and lemmas), the VCs generated and proved, total proof time, and source line count for both abstract and concrete modules. The results were obtained on an Intel Ultra 7 165H, with 16 cores and 64GB of RAM.

Table 1. Report on Verified Operators: Number of Functions (**F**), Ghost Functions (**GF**), Let Functions (**LF**), Requires (**R**) and Ensures (**E**) clauses, Asserts (**A**), Invariants (**I**), Lemmas (**L**) and **VCs** proved, Proof **Time** in seconds, Line count (**LOC**).

Module	F	GF	LF	R	E	A	I	L	VCs	Time (s)	LOC
OPFlatten	3	3	0	12	3	0	0	3	7	0.34	101
COPFlatten	0	0	1	6	1	1	1	0	10	1.23	32
OPAdd	1	1	0	1	0	0	0	0	1	0.02	21
COPAdd	0	0	1	4	1	1	1	0	1	0.52	26
OPSigmoid	1	1	0	0	0	0	0	0	1	0.02	20
COPSigmoid	0	0	1	3	1	1	1	0	11	0.73	25
OPReshape	4	1	0	8	4	0	0	1	2	0.20	107
COPReshape	0	0	1	11	1	1	1	0	19	9.46	62
OPMatmul	1	2	0	4	1	0	0	1	3	0.31	55
COPMatmul	0	0	1	14	2	18	6	1	111	70.42	157
OPConv2d	3	4	0	11	4	0	0	4	11	2.32	145
COPConv2d	0	0	5	74	7	30	22	2	271	278.47	605
OPMaxPool2d	3	3	0	9	3	1	0	3	11	8.2	141
COPMaxpool2d	0	0	4	47	5	25	20	1	194	87.47	531

The data reveal substantial variation in complexity. Element-wise operators such as `Add` and `Sigmoid` are minimal at both levels: `Add` requires a single verification condition at the concrete level, discharged in half a second. Shape manipulation operators `Flatten` and `Reshape` show moderate complexity but remain tractable, with proof times under 10 s. At the other extreme, `Conv2d` represents the most complex development to date, with 271 VCs and about 5 min of cumulative proof time for the concrete implementation, reflecting the nested loop structure, multi-dimensional index arithmetic, stride and padding logic. Across

[3] https://github.com/criticalsoftware/SAIV26-SONNX.

all operators, the concrete layer accounts for the bulk of the verification effort: refinement proofs require establishing pointwise equality through loop invariants, with the nested structure directly impacting proof complexity.

10 Verification of a Complete Model

The formal library and operator specifications developed in Sect. 6 support not only the verified extraction of reference C implementations, but also reasoning at the abstract level about functional properties of complete neural networks. In this section we illustrate this idea taking as example a *logistic regression* model. The model is deliberately simple, but sufficiently rich to demonstrate how operator specifications compose into a verifiable end-to-end network, and how mathematically interesting properties of the model can be stated as WhyML contracts and discharged by automated provers.

The Model. A logistic regression classifies an input by computing a probability value through three successive operations: a matrix multiplication of the input with a weight matrix, addition of a bias term, and application of the sigmoid activation function σ. Given an input tensor X, a weight tensor W, and a bias tensor B, the model computes $net(X, W, B) = \sigma(X \cdot W + B)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$. Figure 3 shows the computation graph. The model is restricted to the case where X has shape $(1, N)$, W has shape $(N, 1)$, and B has shape $(1, 1)$, so that the output (a $(1, 1)$ tensor) represents a class probability.

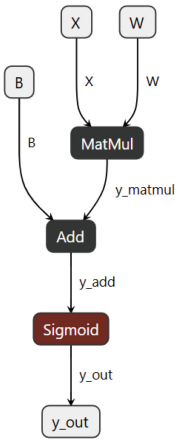


Fig. 3. Computation graph of the logistic regression model

Operator Specifications. Each of the three operators is specified as a ghost function over abstract tensors, following the conventions exemplified for the `Flatten` specification. The matrix multiplication operator `matmul` (Listing 10.1) uses a function `dmatmul` to calculate the data in the result tensor. The latter function is in turn defined using a recursive `dot_product` function that computes the inner product of a row of A and a column of B. The `add` operator performs element-wise addition, and `sigmoid` applies the scalar sigmoid function to each element. When using operator specifications as building blocks to construct networks, it is useful to write postconditions for the operator ghost functions, to facilitate contract-based reasoning about those networks (as exemplified for `matmul` in Listing 10.1). The module includes also some supporting lemmas (see repository). Overall, a total of 13 proof obligations (before splitting) are generated and proved for the three operators.

```
let ghost function matmul ( a b: tensor real) : tensor real
  requires { a.background = b.background }
  requires { length a.dims = length b.dims = 2 }
  requires { get_dim a.dims 1 = get_dim b.dims 0 }
  ensures  { result.background = a.background }
  ensures  { result.dims = Cons (get_dim a.dims 0) (Cons (get_dim b.dims 1) Nil) }
  ensures  { length result.dims = 2 }
  ensures  { result.data = dmatmul a b (get_dim a.dims 0) (get_dim b.dims 1) }
=
  { dims = Cons (get_dim a.dims 0) (Cons (get_dim b.dims 1) Nil) ;
    data = dmatmul a b (get_dim a.dims 0) (get_dim b.dims 1) ;
    background = a.background }
```

Listing 10.1. Matrix multiplication operator

Network-Level Properties. The network is expressed as a ghost function composing the three operators (Listing 10.2). Its contract states several properties of the model, that are then verified by Why3's automated provers. The first postcondition ensures that every output value lies strictly between 0 and 1, as expected of a probability. The second asserts that the output has shape $(1, 1)$. The third states the *neutral case*: when all weights and biases are zero, the output is $\sigma(0) = 0.5$, reflecting the absence of learned information. Beyond the contract of the network function itself, several properties are stated and proved as separate lemmas in this module: *determinism* (given equal inputs, the network produces equal outputs); *controlled symmetry* (negating the weights and bias produces the complementary probability); and *Lipschitz continuity*. Determinism is a direct consequence of the functional nature of all functions involved, and controlled symmetry results from a series of lemmas, in particular antisymmetry of the sigmoid, $(\sigma(-z) + \sigma(z) = 1)$.

Existing work on the computation of Lipschitz constants for neural networks varies both in the methodologies employed to derive such constants and in the degree of expressiveness of the resulting bounds. Research by Christian Szegedy and colleagues [22] estimates the Lipschitz constant of a neural network through the product of the Lipschitz constants of its individual layers, yielding a compositional layer-wise bound. Recent approaches [21] aim at deriving tighter and

```
let ghost function logisticRegression (x w b : tensor real) : tensor real
  requires { lr_constraints x w b }
  (* Output range *)
  ensures { forall ks. valid ks result.dims -> result.data ks > 0.0 /\ result.data ks < 1.0 }
  (* Output shape *)
  ensures { result.dims = Cons 1 (Cons 1 Nil) }
  (* Neutral case *)
  ensures { (forall ks. valid ks w.dims -> w.data ks = 0.0) /\
            (forall ks. valid ks b.dims -> b.data ks = 0.0) ->
            forall ks1. valid ks1 result.dims -> result.data ks1 = 0.5 }
=
  let y_matmul = matmul x w in
  let y_add = add y_matmul b in
  sigmoid y_add

lemma determinism:
  forall x w b x1 w1 b1. x == x1 /\ w == w1 /\ b == b1 ->
    logisticRegression x w b = logisticRegression x1 w1 b1
lemma controlled-symmetry:
  forall x w b. lr_constraints x w b ->
  (logisticRegression x (negativeTensor w) (negativeTensor b)).data (Cons 0 (Cons 0 Nil))
   = 1.0 - (logisticRegression x w b).data (Cons 0 (Cons 0 Nil))
lemma lipschitz-continuity:
  forall x x1 w b. lr_constraints x w b /\ lr_constraints x1 w b ->
    let lr1 = logisticRegression x w b in let lr2 = logisticRegression x1 w b in
    let ks = Cons 0 (Cons 0 Nil) in
    let y1 = lr1.data ks in let y2 = lr2.data ks in
    let z1 = (add (matmul x w) b).data ks in let z2 = (add (matmul x1 w) b).data ks in
    abs (y1 - y2) <= 0.25 * abs (z1 - z2))
```

Listing 10.2. Logistic regression: network definition and contract

more informative bounds by focusing on local Lipschitz continuity. In particular, several works compute input-dependent local Lipschitz constants, which provide robustness guarantees only within specific perturbation regions around selected inputs.

The work we have conducted, algthough being aligned with existing research on the area, focus on operator-wise Lipschitz analysis rather than on layers or entire networks. More specifically, the derivation focuses on the sensitivity of the Sigmoid operator, exploiting its mathematical properties to obtain an exact bound on the network's perturbation sensitivity. From this derivation, it follows that, for any two inputs X_1 and X_2 with the same weights and bias, $|net(X_1, W, B) - net(X_2, W, B)| \leq 0.25 \cdot |z_1 - z_2|$, where $z_i = X_i \cdot W + B$. The bound 0.25 is the maximum of the sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, attained at $z = 0$. This property ensures robustness to input perturbations, providing formal guarantees about output stability under bounded input variations, which is a key requirement for safety certification. The proof uses an axiomatized version of the mean value theorem for the sigmoid function. On the one hand, the approach is fully interpretable and grounded in rigorous mathematical results, enabling a precise and extendible perturbation sensitivity computation. Furthermore, unlike many approximate robustness estimation techniques, the resulting bound is exact. On the other hand, the approach remains limited in scope due to its simplicity. In particular, it does not address the challenges associated with scaling Lipschitz analysis to deep and highly complex neural networks, where layer interactions, activation patterns, and the accumulation of conservative bounds often make exact computation intractable.

The above properties show how the existing formal model can be leveraged to verify graph-level properties; however, this is a representative example rather than the core of the paper.

Overall, a total of 9 proof obligations (before splitting) are generated and proved for the contracts and lemmas concerning the logistic regression net.

11 Limitations

The methodology and workflow presented in this paper carry a number of limitations that should be understood when considering the guarantees provided.

Reasoning over Reals Versus Floating-Point Numbers. All specifications and proofs in this work are carried out over the mathematical reals ($\mathbb{R}$). This choice greatly simplifies reasoning: real arithmetic enjoys nice algebraic properties and is well supported by SMT solvers, which decide useful fragments efficiently. However, the C code extracted by Why3 maps real-valued computations to `float` (or `double`) operations, with rounding introduced at every step. The refinement proofs guarantee that the concrete WhyML implementation computes exactly the same mathematical function as the abstract specification, but this guarantee does not extend past the extraction boundary: the compiled C code only approximates the verified function.

For safety-critical certification, this gap should be addressed by complementary means. One direction is to extend the Why3 development to use the floating-point theories available in the Why3 standard library and in SMT solvers, bounding the rounding error at each operation. Tools such as Gappa [5] and Flocq [2] can assist in this process. Another direction that is being considered in the SONNX working group is to perform a separate numerical analysis on the extracted C code using abstract interpretation, and combine its results with the functional correctness guarantees provided by the current workflow.

Trusted Computing Base. The formal guarantees rest on a trusted computing base that includes several components. First, the Why3 kernel generates VCs from the annotated WhyML code and manages the interaction with external provers. It is a relatively small and well-tested code base, but is not itself formally verified, although recent work has formalized the semantics of core Why3 in Coq and proved two of its internal transformations sound [4], a first step toward a verified implementation. Second, the SMT solver s used to discharge verification conditions are complex pieces of software whose soundness is assumed. In practice, Why3 mitigates this risk by allowing each VC to be checked by multiple independent solvers (e.g. Alt-Ergo, CVC5, and Z3), and by supporting certified backends such as Coq/Rocq. Third, and most relevant to our work, the extraction mechanism that translates verified WhyML code into C is driver-based and not proven sound. This means that it is possible, in principle, for the extracted code to diverge from the verified source, a known limitation that is shared by most extraction-based verification workflows. The risk is mitigated by

the simplicity of the generated C code and by the fact that the reference implementation is intended for back-to-back testing rather than direct deployment, so discrepancies would likely be detected during comparison with production runtimes.

Scope and Scalability. The current operator coverage, while including the main categories of ONNX operators (shape manipulation, linear algebra, convolution), does not yet include all operators required by common architectures. Additionally, the concrete implementations follow a straightforward loop structure chosen for ease of verification rather than performance: the extracted C code is intended as a reference oracle, and is not competitive with optimized production runtimes. Scaling the approach to larger networks will require attention to proof automation, and to the engineering cost of developing new operator specifications.

12 Conclusion

This paper has presented a formally verified workflow for developing tensor operators from abstract specification to reference C implementation, realized in Why3 and supported by a reusable WhyML library. It addresses the need for unambiguous specification and verified implementation articulated by emerging certification standards in safety-critical domains. We have demonstrated the methodology on operators developed within the SONNX Working Group, spanning element-wise operations, shape manipulation, linear algebra, and convolution. Despite substantial variation in complexity, all operators follow the same methodological pattern and extract to clean C code suitable for back-to-back testing. These developments demonstrate that the workflow scales from trivial operators to complex ones, and that the reusable library infrastructure supports the entire spectrum of operator types encountered in practice.

In Sect. 10 we have shown how the abstract tensor library and operator specifications can be composed to reason about network-level properties, without any involvement of the concrete layer or C code. The verified properties are purely mathematical, expressed and proved over $\mathbb{R}$, abstracting rounding errors, and concern the ideal behavior of the model rather than the numerical behavior of a specific implementation. This approach has notable limitations in its current form: in the example, the mean value theorem for the sigmoid was introduced as an axiom rather than proved within Why3. Proving such results would require either extending the Why3 standard library or interfacing with an interactive prover such as Coq/Rocq (see [27] for a proof of this theorem).

Despite this, the logistic regression example demonstrates that the same formal infrastructure developed for verified operator extraction can also serve as a foundation for model-level verification – a direction of particular interest for the certification of safety-critical ML systems. The current work already covers the necessary operators for the core of architectures such as MNIST/LeNet-5 [16,19] and AlexNet [15], with only minor additions required for full coverage; we are currently investigating whether the verification approach demonstrated for logistic regression scales effectively to such networks. The limitations of the current

workflow, including the gap between real-arithmetic proofs and floating-point execution, are discussed in Sect. 11.

Acknowledgments. This work is funded by national funds through FCT Fundação para a Ciência e a Tecnologia, I.P., under the support UID/50014/2025(https://doi.org/10.54499/UID/50014/2025.).

References

1. Abadi, M., et al.: TensorFlow: Large-scale machine learning on heterogeneous systems (2015), software available from tensorflow.org
2. Boldo, S., Melquiond, G.: Flocq: A unified library for proving floating-point algorithms in Coq. In: Proceedings of the 20th IEEE Symposium on Computer Arithmetic (ARITH), pp. 243–252 (2011)
3. Chiu, Y.C., Tsai, C.Y., Ruan, M.D., Shen, G.Y., Lee, T.T.: Mobilenet-ssdv2: an improved object detection model for embedded systems. In: 2020 International Conference on System Science and Engineering (ICSSE), pp. 1–5 (2020)
4. Cohen, J.M., Johnson-Freyd, P.: A Formalization of Core Why3 in Coq. Proc. ACM Program. Lang. **8**(POPL) (2024)
5. Daumas, M., Melquiond, G.: Certification of bounds on expressions involving rounded operators. ACM Trans. Math. Softw. **37**(1), 1–20 (2010)
6. EASA: Concept paper: Guidance for level 1 and 2 machine learning applications. Tech. rep., European Union Aviation Safety Agency (2024), issue 02
7. EASA: Proposed detailed specifications and associated acceptable means of compliance and guidance material for AI trustworthiness (DS.AI). Tech. Rep. NPA 2025-07 (B), European Union Aviation Safety Agency (2025)
8. Ehlers, R.: Formal verification of piece-wise linear feed-forward neural networks. In: D'Souza, D., Narayan Kumar, K. (eds.) Automated Technology for Verification and Analysis, pp. 269–286. Springer International Publishing, Cham (2017)
9. Filliâtre, J.-C., Paskevich, A.: Why3 — Where Programs Meet Provers. In: Felleisen, M., Gardner, P. (eds.) ESOP 2013. LNCS, vol. 7792, pp. 125–128. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-37036-6_8
10. Gabreau, C., Pesquet-Popescu, B., Kaakai, F., Lefevre, B.: AI for future skies: Ongoing standardization activities to build the next certification/approval framework for airborne and ground aeronautical products. In: CEUR Workshop Proceedings, vol. 2916 (2021)
11. Huang, X., et al.: A survey of safety and trustworthiness of deep neural networks: Verification, testing, adversarial attack and defence, and interpretability. Comput. Sci. Rev. **37**, 100270 (2020)
12. Jajal, P., et al.: Interoperability in deep learning: A user survey and failure analysis of ONNX model converters. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24), pp. 1466–1478. ACM, Vienna, Austria (2024)
13. Jenn, E., et al.: SONNX: Towards an ONNX profile for critical systems. In: European Congress of Embedded Real Time Systems (ERTS) (2026), iSSN 2680-0918
14. Katz, G., Barrett, C., Dill, D.L., Julian, K., Kochenderfer, M.J.: Reluplex: An efficient smt solver for verifying deep neural networks. In: Majumdar, R., Kunčak, V. (eds.) Computer Aided Verification, pp. 97–117. Springer International Publishing, Cham (2017)

15. Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. In: Advances in Neural Information Processing Systems 25 (NeurIPS 2012), pp. 1097–1105 (2012)
16. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proc. IEEE **86**(11), 2278–2324 (1998)
17. Liu, C., Arnon, T., Lazarus, C., Strong, C., Barrett, C.W., Kochenderfer, M.J.: Algorithms for verifying deep neural networks. Found. Trends Optim. **4**(3–4), 244–400 (2021)
18. Melquiond, G., Rieu-Helft, R.: WhyMP, a formally verified arbitrary-precision integer library. J. Symb. Comput. **115**, 74–95 (2023)
19. MNIST - Handwritten Digit Recognition ONNX model. https://github.com/onnx/models/tree/main/validated/vision/classification/mnist
20. ONNX Community: ONNX: Open neural network exchange (2024). https://onnx.ai/
21. Shi, Z., Wang, Y., Zhang, H., Kolter, Z., Hsieh, C.J.: Efficiently computing local lipschitz constants of neural networks via bound propagation (2022)
22. Szegedy, C., et al.: Intriguing properties of neural networks (2014)
23. The Khronos Group: Neural network exchange format (NNEF) (2024). https://www.khronos.org/nnef/
24. Wu, H., et al.: Marabou 2.0: A versatile formal analyzer of neural networks. In: Gurfinkel, A., Ganesh, V. (eds.) Computer Aided Verification, pp. 249–264. Springer, Cham (2024)
25. Zhang, H., Weng, T.W., Chen, P.Y., Hsieh, C.J., Daniel, L.: Efficient neural network robustness certification with general activation functions. In: Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., Garnett, R. (eds.) Advances in Neural Information Processing Systems, vol. 31. Curran Associates, Inc. (2018). https://proceedings.neurips.cc/paper_files/paper/2018/file/d04863f100d59b3eb688a11f95b0ae60-Paper.pdf
26. Zhang, J., Liu, J., Dong, Y., Su, H.: A survey of neural network robustness assessment in image recognition. arXiv preprint arXiv:2404.08285 (2024)
27. Zhou, Y., Gao, C., Yu, W.: Formal proof of the mean value theorem based on Coq. In: 2023 China Automation Congress (CAC), pp. 5726–5730 (2023)